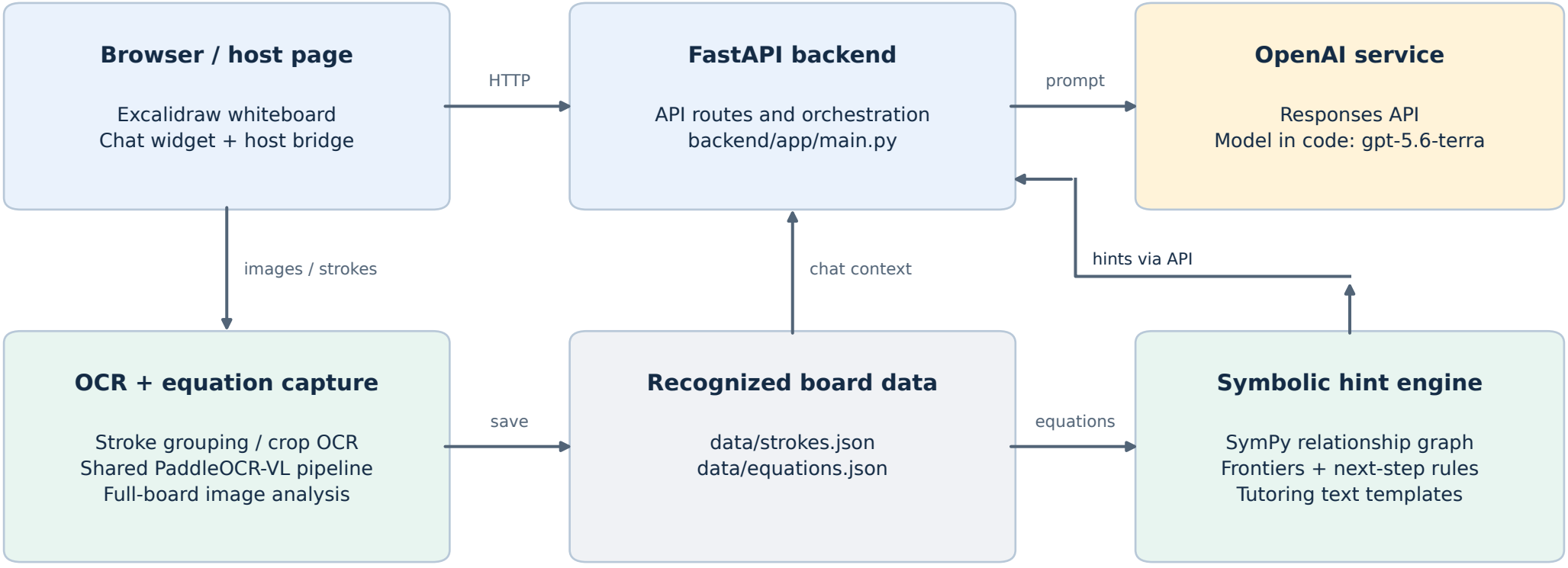
Agent architecture

A whiteboard tutoring application with OCR, contextual chat, and symbolic algebra hints.



Arrows show request and data dependencies; API responses return to the calling client.

Supporting state and lifecycle

In memory: current scene, OCR analysis, bounded analysis history, and room presence.
On disk: equation/stroke snapshots, OpenAI request logs, and whiteboard action logs.

How a question becomes an answer

The chat route combines recognized equations with optional full-board OCR.

1 Capture the question

The widget asks the host bridge to export the board as PNG.
It posts the question and a PNG, when available, to POST /api/chat.

2 Read the current board

If an image is supplied, PaddleOCR-VL analyzes it and the backend saves
the analysis in memory. The route also accepts optional scene_json.

3 Assemble text context

equation_context.py reads non-empty equations from data/equations.json,
sorts them by board position, and includes equation IDs and contributors.

4 Generate the response

The route combines equation context with formatted full-board analysis.
openai_service.py sends the question and context to the OpenAI Responses API.

5 Return and display

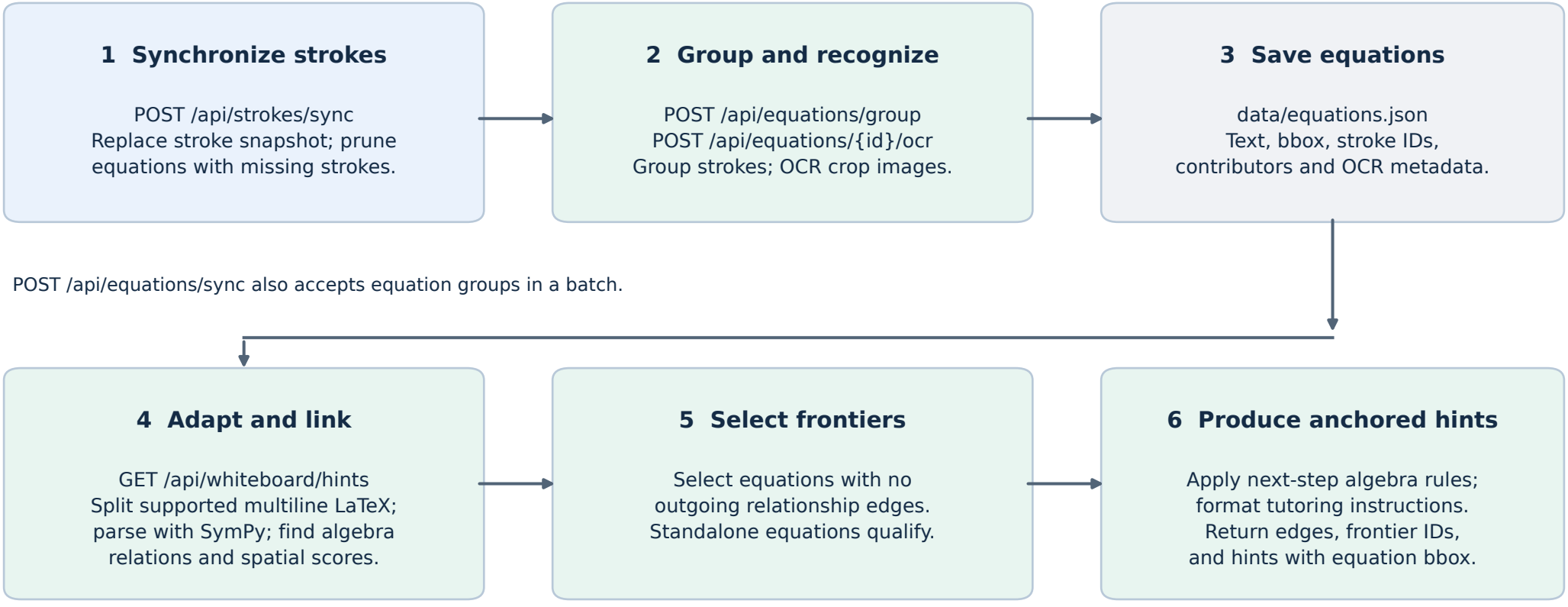
The backend logs the model request/response and returns question, answer,
and analysis. The widget renders Markdown and sanitizes it with DOMPurify.

Implementation boundaries

Each model call uses the current question and board context; previous chat turns are not passed.
OCR runs locally through PaddleOCR-VL; the OpenAI call receives assembled text.
The chat route does not invoke the symbolic hint pipeline or run an autonomous tool loop.

Equation memory and tutoring hints

data/equations.json is the shared input to both chat context and symbolic reasoning.



Scope of the current implementation

Hint rules target simple one-variable linear equations, with the variable on the left.
The supplied widget directly calls /api/chat; equation-sync and hint endpoints are exposed for host integration.
Room presence tracks one room in memory and can reset board snapshots when the room becomes empty.

Source map (paths relative to the repository)

widget/src/ui/chat/messages.js | backend/app/api/routes/{chat,equations,strokes}.py
backend/app/services/{equation_context,equation_ocr,openai_service,room_presence}.py
backend/app/api/whiteboard.py | backend/app/reasoning/{pipeline,graph,frontier,next_step}.py